

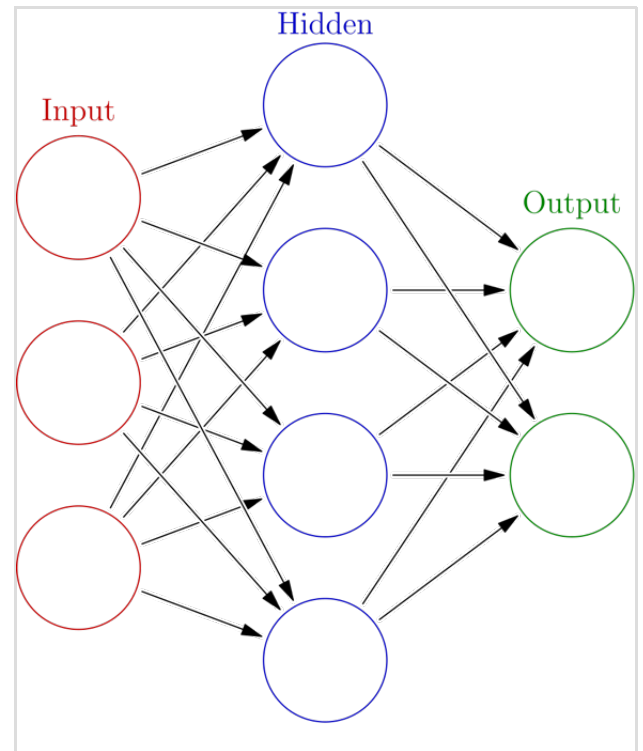
Neural network (machine learning)

In machine learning, a **neural network** (NN) or **neural net**, also known as an **artificial neural network** (ANN), is a computational model inspired by the structure and functions of biological neural networks.^{[1][2]}

A neural network consists of connected units or nodes called artificial neurons, which loosely model the neurons in the brain. Artificial neuron models that mimic biological neurons more closely have also been recently investigated and shown to significantly improve performance. These are connected by *edges*, which model the synapses in the brain. Each artificial neuron receives signals from connected neurons, then processes them and sends a signal to other connected neurons. The "signal" is a real number, and the output of each neuron is computed by some non-linear function of the totality of its inputs, called the activation function. The strength of the signal at each connection is determined by a *weight*, which adjusts during the learning process.

Typically, neurons are aggregated into layers. Different layers may perform different transformations on their inputs. Signals travel from the first layer (the *input layer*) to the last layer (the *output layer*), possibly passing through multiple intermediate layers (*hidden layers*). A network is typically called a deep neural network if it has at least two hidden layers.^[3]

Advances in computing power, particularly the use of graphics processing units (GPUs), and the availability of large datasets further accelerated neural network research in the early 21st century. These developments enabled the training of deep neural networks capable of learning hierarchical representations from complex data.



An artificial neural network is an interconnected group of nodes, inspired by a simplification of neurons in a brain. Here, each blue/green circular node in the hidden and output layers represents an artificial neuron and each red circular node in the far left layer represents an input data value. An arrow represents a connection from the output of one neuron (or data node) to the input of another.

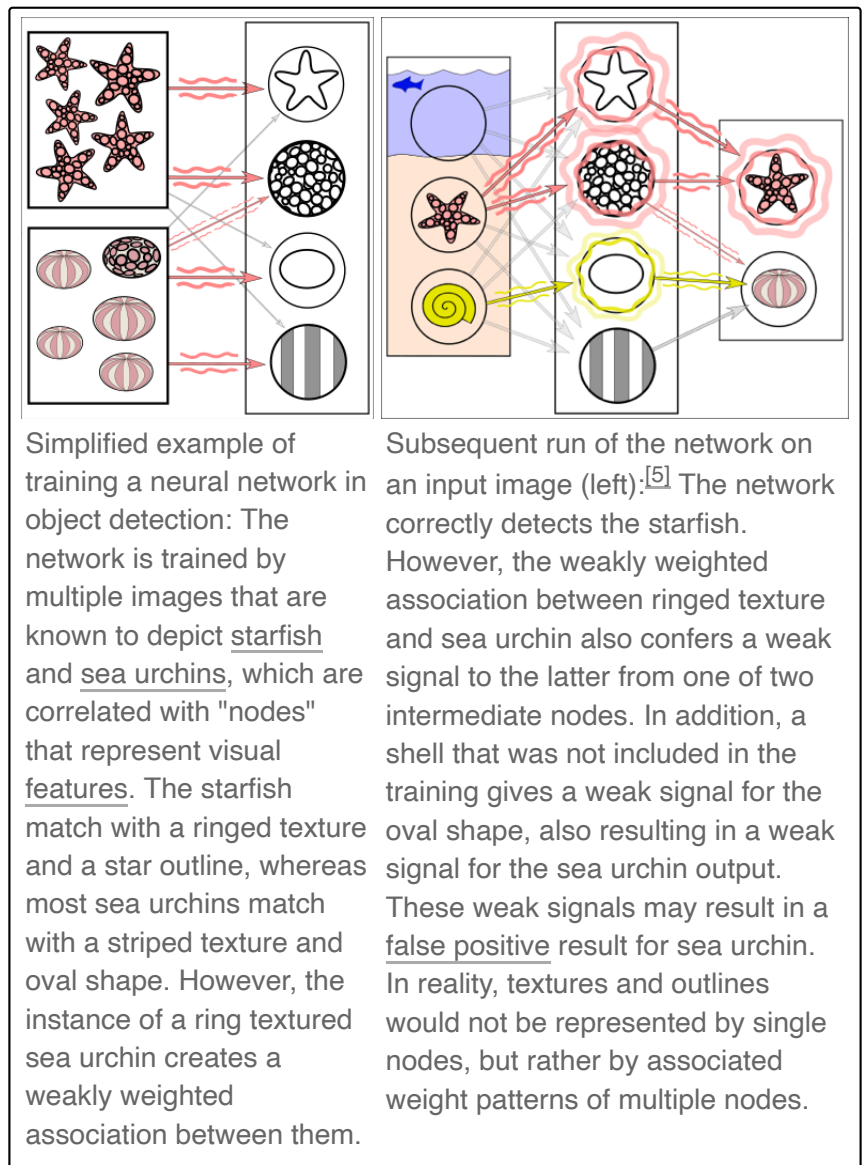
Architectural innovations such as convolutional neural networks (CNNs) significantly improved performance in computer vision tasks, while recurrent neural networks (RNNs) enabled modeling of sequential data such as speech and time-series information. More recently, transformer architectures introduced attention mechanisms that allow neural networks to model long-range dependencies in data and have become foundational for modern large language models.^[4]

Today, artificial neural networks are used for various tasks, including predictive modeling, adaptive control, and solving problems in artificial intelligence.

History

Mathematical foundations

Today's deep neural networks are based on early work in statistics over 200 years ago. The simplest kind of feedforward neural network (FNN) is a linear network, which consists of a single layer of output nodes with linear activation functions; the inputs are fed directly to the outputs via a series of weights. The sum of the products of the weights and the inputs is calculated at each node. The mean squared errors between these calculated outputs and the given target values are minimized by creating an adjustment to the weights. This technique has been known for over two centuries as the method of least squares or linear regression. It was used as a means of finding a good rough linear fit to a set of points by Legendre (1805) and Gauss (1795) for the prediction of planetary movement.^{[6][7][8][9][10]}



Perceptrons

Historically, digital computers such as the von Neumann model operate via the execution of explicit instructions with access to memory by a number of processors. Some neural networks, on the other hand, originated from efforts to model information processing in biological systems through the framework of connectionism. Unlike the von Neumann model, connectionist computing does not separate memory and processing.

Warren McCulloch and Walter Pitts^[11] (1943) considered a non-learning computational model for neural networks.^[12] This model paved the way for research to split into two approaches; one approach focused on biological processes while the other focused on the application of neural networks to artificial intelligence. McCulloch and Pitts also developed mathematical models of artificial neurons capable of representing logical functions.^[11]

In the late 1940s, D. O. Hebb^[13] proposed a learning hypothesis based on the mechanism of neural plasticity that became known as Hebbian learning. It was used in many early neural networks, such as Rosenblatt's perceptron and the Hopfield network. Farley and Clark^[14] (1954) used computational machines to simulate a Hebbian network. Other neural network computational machines were created by Rochester, Holland, Habit and Duda (1956).^[15]

In 1958, psychologist Frank Rosenblatt described the perceptron, one of the first implemented artificial neural networks,^{[16][17][18][19]} funded by the United States Office of Naval Research.^[20] R. D. Joseph (1960)^[21] mentions an even earlier perceptron-like device by B. G. Farley and W. A. Clark of the MIT Lincoln Laboratory;^[9] however, according to Joseph, "they dropped the subject."^[21]

The first perceptrons did not have adaptive hidden units. However, Joseph (1960)^[21] also discussed multilayer perceptrons with an adaptive hidden layer. Rosenblatt (1962)^[22]:section 16 cited and adopted these ideas, also crediting work by H. D. Block and B. W. Knight. Unfortunately, these early efforts did not lead to a working learning algorithm for hidden units, i.e., deep learning.

The perceptron raised public excitement for research in artificial neural networks, causing the US government to drastically increase funding. This contributed to "the Golden Age of AI", fueled by the optimistic claims made by computer scientists regarding the ability of perceptrons to emulate human intelligence.^[23]

Historical foundations and the Dartmouth proposal

Artificial neural networks were identified as a promising direction for artificial intelligence research in the 1955 proposal for the Dartmouth Summer Research Project on Artificial Intelligence.^[24] In the proposal, researchers suggested that simplified computational models of biological neurons, described as "neuron nets," might enable machines to learn, form concepts, and improve performance through experience.^[24]

Despite this early promise, neural network models faced major limitations during the early decades of artificial intelligence research. Hardware constraints limited network size and training efficiency, while theoretical understanding of learning algorithms remained incomplete. Many early models relied on single-layer perceptrons, which were restricted to solving linearly separable problems. These limitations were highlighted in the book *Perceptrons* by Marvin Minsky and Seymour Papert, which contributed to reduced interest in neural network research during the late 1960s and 1970s.^[25]

Deep learning breakthroughs in the 1960s and 1970s

Fundamental research was conducted on ANNs in the 1960s and 1970s. The first working deep learning algorithm was the Group method of data handling, a method to train arbitrarily deep neural networks, published by Alexey Ivakhnenko and Valentin Lapa in the Soviet Union (1965). They regarded it as a form of polynomial regression,^[26] or a generalization of Rosenblatt's perceptron.^[27] A 1971 paper described a deep network with eight layers trained by this method,^[28] which is based on layer by layer training through regression analysis. Superfluous hidden units are pruned using a separate validation set. Since the activation functions of the nodes are Kolmogorov-Gabor polynomials, these were also the first deep networks with multiplicative units or "gates."^[9]

The first deep learning multilayer perceptron (MLP) trained by stochastic gradient descent^[29] was published in 1967 by Shun'ichi Amari.^[30] In computer experiments conducted by Amari's student S. Saito, a five layer MLP with two modifiable layers learned internal representations to classify non-linearly separable pattern classes.^[9] Subsequent developments in hardware and hyperparameter tuning have made end-to-end stochastic gradient descent the currently dominant training technique.

In 1969, Kunihiro Fukushima introduced the ReLU (rectified linear unit) activation function.^{[9][31][32]} The rectifier has become the most popular activation function for deep learning.^[33]

Nevertheless, research stagnated in the United States following the work of Minsky and Papert (1969),^[34] who emphasized that basic perceptrons were incapable of processing the exclusive-or circuit. This insight was irrelevant for the deep networks of Ivakhnenko (1965) and Amari (1967).

In 1976, transfer learning was introduced.^[35]

Backpropagation

Interest in neural networks revived during the 1980s with the development of the backpropagation algorithm, which allowed multi-layer neural networks to be trained efficiently by propagating error gradients backward through network layers.^[36] Backpropagation is an efficient application of the chain rule derived by Gottfried Wilhelm Leibniz in 1673^[37] to networks of differentiable nodes. The

terminology "back-propagating errors" was actually introduced in 1962 by Rosenblatt,^[22] but he did not know how to implement this, although Henry J. Kelley had a continuous precursor of backpropagation in 1960 in the context of control theory.^[38] In 1970, Seppo Linnainmaa published the modern form of backpropagation in his master's thesis (1970).^{[39][40][9]} G.M. Ostrovski et al. republished it in 1971.^{[41][42]} Paul Werbos applied backpropagation to neural networks in 1982^[43]^[44] (his 1974 PhD thesis, reprinted in a 1994 book,^[45] did not yet describe the algorithm^[42]). In 1986, David E. Rumelhart et al. popularised backpropagation but did not cite the original work.^[46]

Convolutional neural networks

Deep learning architectures for convolutional neural networks (CNNs) with convolutional layers and downsampling layers and weight replication began with the neocognitron introduced by Kunihiro Fukushima in 1979, though not trained by backpropagation.^{[47][48][49]} Fukushima's CNN architecture also introduced max pooling,^[50] a popular downsampling procedure for CNNs. CNNs have become an essential tool for computer vision.

The time delay neural network (TDNN) was introduced in 1987 by Alex Waibel to apply CNNs to phoneme recognition. It used convolutions, weight sharing, and backpropagation.^{[51][52]} In 1988, Wei Zhang applied a backpropagation-trained CNN to alphabet recognition.^[53] In 1989, Yann LeCun et al. created a CNN called LeNet for recognizing handwritten ZIP codes on mail. Training required 3 days.^[54] In 1990, Wei Zhang implemented a CNN on optical computing hardware.^[55] In 1991, a CNN was applied to medical image object segmentation^[56] and breast cancer detection in mammograms.^[57] LeNet-5 (1998), a 7-level CNN by Yann LeCun et al. that classifies digits, was applied by several banks to recognize hand-written numbers on checks digitized in 32×32 pixel images.^[58]

From 1988 onward,^{[59][60]} the use of neural networks transformed the field of protein structure prediction, in particular when the first cascading networks were trained on *profiles* (matrices) produced by multiple sequence alignments.^[61]

Recurrent neural networks

One origin of RNN was statistical mechanics. In 1972, Shun'ichi Amari proposed to modify the weights of an Ising model by Hebbian learning rule as a model of associative memory, adding in the component of learning.^[62] This was popularized as the Hopfield network by John Hopfield (1982).^[63] Another origin of RNN was neuroscience. The word "recurrent" is used to describe loop-like structures in anatomy. In 1901, Cajal observed "recurrent semicircles" in the cerebellar cortex.^[64] Hebb considered "reverberating circuit" as an explanation for short-term memory.^[65] The McCulloch and Pitts paper (1943) considered neural networks that contain cycles, and noted that the current activity of such networks can be affected by activity indefinitely far in the past.^[11]

In 1982 a recurrent neural network with an array architecture (rather than a multilayer perceptron architecture), namely a Crossbar Adaptive Array,^{[66][67]} used direct recurrent connections from the output to the supervisor (teaching) inputs. In addition of computing actions (decisions), it computed internal state evaluations (emotions) of the consequence situations. Eliminating the external supervisor, it introduced the self-learning method in neural networks.

In cognitive psychology, the journal American Psychologist in early 1980s carried out a debate on the relation between cognition and emotion. Social psychologist Robert Zajonc in 1980 stated that emotion is computed first and is independent from cognition, while Richard Lazarus in 1982 stated that cognition is computed first and is inseparable from emotion.^{[68][69]} In 1982 the Crossbar Adaptive Array gave a neural network model of cognition-emotion relation.^{[66][70]} It was an example of a debate where an AI system, a recurrent neural network, contributed to an issue in the same time addressed by cognitive psychology.

Two early influential works were the Jordan network (1986) and the Elman network (1990), which applied RNN to study cognitive psychology.

In the 1980s, backpropagation did not work well for deep RNNs. To overcome this problem, in 1991, Jürgen Schmidhuber proposed the "neural sequence chunker" or "neural history compressor"^{[71][72]} which introduced the important concepts of self-supervised pre-training (the "P" in ChatGPT) and neural knowledge distillation.^[9] In 1993, a neural history compressor system solved a "Very Deep Learning" task that required more than 1000 subsequent layers in an RNN unfolded in time.^[73]

In 1991, Sepp Hochreiter's diploma thesis identified and analyzed the vanishing gradient problem^{[74][75]} and proposed recurrent residual connections to solve it. He and Schmidhuber introduced long short-term memory (LSTM), which set accuracy records in multiple applications domains.^{[76][77]} This was not yet the modern version of LSTM, which required the forget gate, which was introduced in 1999.^[78] It became the default choice for RNN architecture.

During 1985–1995, inspired by statistical mechanics, several architectures and methods were developed by Terry Sejnowski, Peter Dayan, Geoffrey Hinton, and others, including the Boltzmann machine,^[79] restricted Boltzmann machine,^[80] Helmholtz machine,^[81] and the wake-sleep algorithm.^[82] These were designed for unsupervised learning of deep generative models.

Modern deep learning

Between 2009 and 2012, ANNs began winning prizes in image recognition contests, approaching human level performance on various tasks, initially in pattern recognition and handwriting recognition.^{[83][84]} In 2011, a CNN named *DanNet*^{[85][86]} by Dan Ciresan, Ueli Meier, Jonathan Masci, Luca Maria Gambardella, and Jürgen Schmidhuber achieved for the first time superhuman

performance in a visual pattern recognition contest, outperforming traditional methods by a factor of 3.^[49] It then won more contests.^{[87][88]} They also showed how max-pooling CNNs on GPU improved performance significantly.^[89]

In October 2012, AlexNet by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton^[90] won the large-scale ImageNet competition by a significant margin over shallow machine learning methods. Further incremental improvements included the VGG-16 network by Karen Simonyan and Andrew Zisserman^[91] and Google's Inceptionv3.^[92]

In 2012, Ng and Dean created a network that learned to recognize higher-level concepts, such as cats, only from watching unlabeled images.^[93] Unsupervised pre-training and increased computing power from GPUs and distributed computing allowed the use of larger networks, particularly in image and visual recognition problems, which became known as "deep learning".^[94]

Radial basis function and wavelet networks were introduced in 2013. These can be shown to offer best approximation properties and have been applied in nonlinear system identification and classification applications.^[95]

Generative adversarial networks (GANs) (Ian Goodfellow et al., 2014)^[96] became state of the art in generative modeling in 2014–2018. The GAN principle was originally published in 1991 by Jürgen Schmidhuber, who called it "artificial curiosity": two neural networks contest with each other in the form of a zero-sum game, where one network's gain is the other network's loss.^{[97][98]} The first network is a generative model that models a probability distribution over output patterns. The second network learns by gradient descent to predict the reactions of the environment to these patterns. Excellent image quality is achieved by Nvidia's StyleGAN (2018)^[99] based on the Progressive GAN by Tero Karras et al.^[100] Here, the GAN generator is grown from small to large scale in a pyramidal fashion. Image generation by GAN reached popular success, and provoked discussions concerning deepfakes.^[101] Diffusion models (2015)^[102] eclipsed GANs in generative modeling since then, with systems such as DALL·E 2 (2022) and Stable Diffusion (2022).

In 2014, the state of the art was training "[a] very deep neural network" with 20 to 30 layers.^[103] Stacking too many layers led to a steep reduction in training accuracy,^[104] known as the "degradation" problem.^[105] In 2015, two techniques were developed to train very deep networks: the highway network was published in May 2015,^[106] and the residual neural network (ResNet) in December 2015.^{[107][108]} ResNet behaves like an open-gated Highway Net.

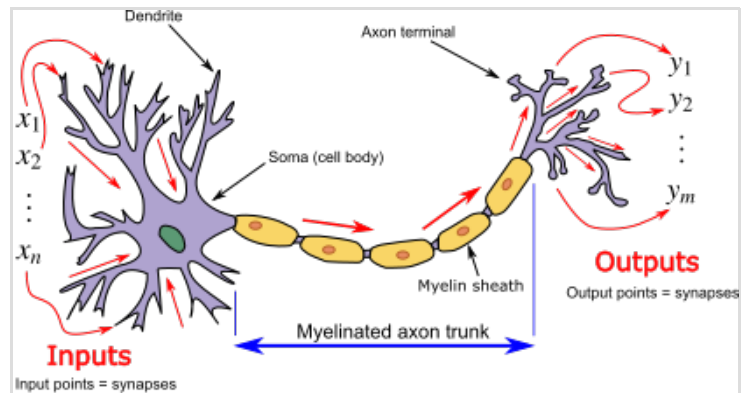
Transformers

During the 2010s, the seq2seq model was developed, and attention mechanisms were added. It led to the modern transformer architecture in 2017 in *Attention Is All You Need*.^[4] It requires computation time that is quadratic in the size of the context window. Jürgen Schmidhuber's fast

weight controller (1992)^[109] scales linearly and was later shown to be equivalent to the unnormalized linear transformer.^{[110][111][9]} Transformers have increasingly become the model of choice for natural language processing.^[112] Many modern large language models such as ChatGPT, GPT-4, and BERT use this architecture.

Models

ANNs began as an attempt to exploit the architecture of the human brain to perform tasks that conventional algorithms had little success with. They soon reoriented towards improving empirical results, abandoning attempts to remain true to their biological precursors. ANNs have the ability to learn and model complex, non-linear relationships. This is achieved by neurons being connected in various patterns, allowing the output of some neurons to become the input of others. The network forms a directed, weighted graph.^[113]

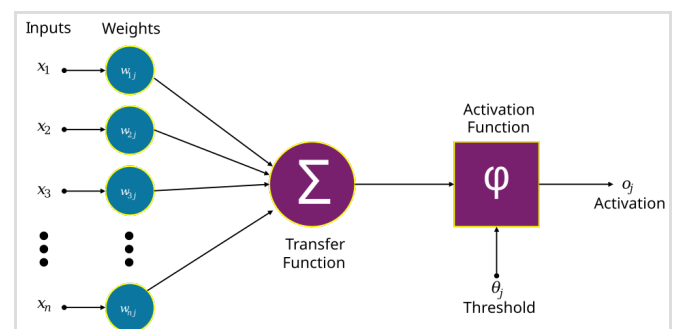


Neuron and myelinated axon, with signal flow from inputs at dendrites to outputs at axon terminals

An artificial neural network consists of simulated neurons. Each neuron is connected to other nodes via links like a biological axon-synapse-dendrite connection. All the nodes connected by links take in some data and use it to perform specific operations and tasks on the data. Each link has a weight, determining the strength of one node's influence on another.^[114]

Artificial neurons

ANNs are composed of artificial neurons, conceptually derived from biological neurons. Each artificial neuron has one or more numerical inputs and produces a single numerical output.^[115] To find the output of a neuron, we take the (weighted) sum of all the neuron's inputs, weighted by the *weights* of the *connections* from each of those inputs. We then add a *bias* term to this sum.^[116] This weighted sum (plus the bias) is then passed through a nonlinear activation function to produce the neuron's output.^{[117][118][119]}

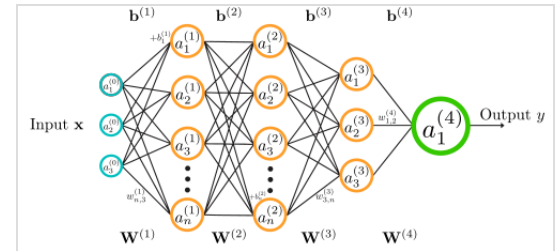


Structure of an artificial neuron

To create a neural network, multiple artificial neurons are connected together, with the outputs of some neurons being fed into the input of others. The inputs to the initial *input neurons* are external data, such as images and text (encoded as a series of numbers). The outputs of the final *output neurons* correspond to the task the network was created to do, such as recognizing an object in an image.^[117]

Organization

The neurons are typically organized into multiple layers, especially in deep learning. The layer that receives external data is the *input layer*, and the layer that produces the ultimate result is the *output layer*. In between them are zero or more *hidden layers*. Single layer and unlayered networks are also used.



Structure of a feedforward neural network with four layers

Between two layers, multiple connection patterns are possible. Traditionally, they are *fully connected*, with every neuron in one layer connecting to every neuron in the next layer.^[120] However, in convolutional neural networks, some layers are *convolutional*, meaning each neuron in the next layer is connected to a set of spatially grouped neurons in the previous layer.^{[121][122]}

In most neural networks, the outputs of the neurons in one layer are only connected to the inputs of neurons in the immediately following layer (forming a directed acyclic graph), meaning information can only flow "forward" from one layer to the next. These are known as feedforward networks.^[123] In contrast, networks that allow connections between neurons in the same or previous layers are known as recurrent networks.^[124]

Learning

The function of a neural network is defined by its architecture – how the different neurons are organized and connected to one another – and the values, or *weights*, associated with each of the neurons, determining how it calculates its output based on its input connections.^{[125][note 1]} The goal of learning involves adjusting the weights (and optional thresholds) of the network to improve the accuracy of the result. This is done by minimizing the observed errors among sample observations.

Neural networks are typically trained through empirical risk minimization, which is based on the idea of optimizing the network's parameters to minimize the difference, or empirical risk, between the predicted output and the actual target values in a given dataset.^[126] Gradient-based methods

such as backpropagation are usually used to estimate the parameters of the network.^[126] During the training phase, ANNs learn from labeled training data by iteratively updating their parameters to minimize a defined loss function.^[94] This method allows the network to generalize to unseen data.

Practically this is done by defining a loss function^[note 2] that is evaluated periodically during learning. As long as its output continues to decline, learning continues. The cost is frequently defined as a statistic whose value can only be approximated. The outputs are actually numbers, so when the error is low, the difference between the output (almost certainly a cat) and the correct answer (cat) is small. Learning attempts to reduce the total of the differences across the observations. Most learning models can be viewed as a straightforward application of optimization theory and statistical estimation.^{[113][127]}

Learning is typically considered complete when examining additional observations does not usefully reduce the error rate. Even after learning, the error rate typically does not reach 0. If after learning, the error rate is too high, the network typically must be redesigned.

Loss function

While it is possible to define a loss function ad hoc, frequently the choice is determined by the function's desirable properties (such as convexity, differentiability, and robustness)^[128] because it arises from the model (e.g. in a probabilistic model, the model's posterior probability can be used as an inverse cost).

Backpropagation

Backpropagation is a method used to adjust the connection weights to compensate for each error found during learning. The error amount is effectively divided among the connections. Technically, backpropagation calculates the gradient (the derivative) of the loss function associated with a given state with respect to the weights. The weight updates can be done via stochastic gradient descent or other methods, such as extreme learning machines,^[129] "no-prop" networks,^[130] training without backtracking,^[131] "weightless" networks,^{[132][133]} and non-connectionist neural networks.

Hyperparameters

A hyperparameter is a parameter defining any configurable part of the learning process, whose value is set prior to training.^[134] Examples of hyperparameters include learning rate, batch size and regularization parameters.^[135] The performance of a neural network is strongly influenced by the choice of hyperparameter values, and thus the hyperparameters are often optimized as part of the training process, a process called hyperparameter tuning or hyperparameter optimization.^[136]